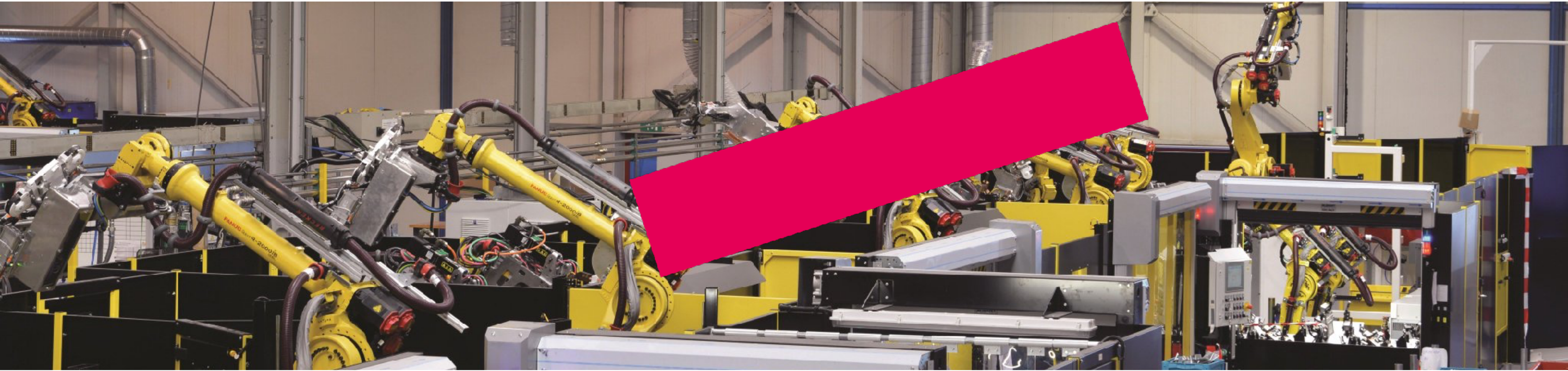


Programming 6



Workshop on Clean Code

johan.korten@han.nl

V1.0 Nov 2025

Workshop Topics overview

Clean code (beyond SOLID)

CMake

Unit testing

Commenting

Patterns (Clean architecture)

A few reasons why do we need clean code...

All software:

- Maintainability (Code is read 10x more than written)
- Collaboration (Teams need shared understanding)

Mr. Clean Code: Robert C. Martin aka Uncle Bob

<https://gist.github.com/wojteklu/73c6914cc446146b8b533c0988cf8d29>

Clean code: Maintainability

How do we solve maintainability and collaboration mostly?

Clean code: Maintainability

Clean code versus Clean architecture

A few reasons why do we need clean code...

All software:

- Maintainability (Code is read 10x more than written)
- Collaboration (Teams need shared understanding)
- Especially embedded:
 - Safety - Lives depend on it (medical, automotive, aerospace)
 - Certification - IEC 62304, ISO 26262, DO-178C compliance required
 - Resource constraints - Limited RAM/ROM, no dynamic allocation
 - Longevity - Products run 10-20+ years unchanged (e.g. MOBA)
 - Debugging difficulty - No printf, limited tooling, real-time constraints
 - Efficiency - E.g. battery life, etc.

When Code Quality Kills 1/2

Case 1: Therac-25 (1985-1987)

The Incident:

Radiation therapy machine delivered ***doses 100x higher than intended***

At least 6 accidents between 1985-1987

3 patients died, others severely injured

Code Problems:

- Race condition: changing treatment mode within 8 seconds left turntable in wrong position
- One-byte flag overflow: every 256th increment reset safety check to zero
- Hardware safety interlocks removed, relying solely on software
- Reused code from previous machines (Therac-6, Therac-20) without hardware safeguards
- Cryptic error messages ("MALFUNCTION 54") with no documentation
- No formal specifications, design documents, or test plans

When Code Quality Kills 2/2

Case 2: Toyota Unintended Acceleration (2000s-2010)

The Incident:

Reports of ***unintended acceleration*** led to investigation

NHTSA estimated **89 deaths** in the prior decade (by 2010)

\$3+ billion in settlements and fines

Multiple recalls involving millions of vehicles

The Code Problems:

- [Error cannot fit on single slide ;)]

When Code Quality Kills 2/2 (Toyota Unintended Acceleration)

Case 2: Toyota Unintended Acceleration (2000s-2010)

The Code Problems (NASA Investigation (2010-2011)):

- Found **7,134** MISRA-C violations (checking only 35 of 127 rules)
- **347** violations of subset of MISRA-C rules they checked
- **243** violations of NASA's own "Power of 10" safety rules
- Concluded: no definitive electronic cause found, but system was highly complex
- Civil Litigation Expert Analysis (2013): Michael Barr's team found in 2005 Camry code:
 - **81,514** MISRA-C violations (checking full 2004 ruleset)
 - **11,000** global variables
 - **67** functions rated "untestable" (cyclomatic complexity >50)
 - Stack overflow risks (**94%** usage, not 41% as Toyota claimed)
 - Critical variables not mirrored for safety
 - No watchdog protection on critical task
 - Recursion in safety-critical code (MISRA violation)
 - Toyota's Standards: Used only 11 of 127 MISRA-C rules; violated 5 of those 11

When Code Quality Kills: Resume

Both cases show:

"It compiles" $\neq$ "It's safe"

Standards exist for reasons - written in blood

Testing alone is insufficient - code must be maintainable and verifiable

Overconfidence kills - both companies dismissed early warnings

These weren't syntax errors. These were design and process failures that standards like MISRA specifically prevent. Let's learn what those standards are and why they matter."

When Code Quality Kills: Resume

From SOLID to safety-critical coding:

- SOLID helps with maintainability, but embedded systems need more...
- Clean code matters differently in embedded: safety, certification, resource constraints

Trade-offs: why *software engineering* vs programming or computer science

Performance vs safety, readability vs efficiency

MISRA

Safety
Correctness
Standards



Prevents Crashes

Code Smells

Design
Maintainability
Clean Architecture



Reduces Tech Debt

Readability

Communication
Understanding
Clarity



Enables Maintenance

MISRA Compliance on C++:

“While popular, no programming language can guarantee that the final executable code will behave exactly as the developer intended.

The language has several drawbacks which are discussed in the following sub-sections.”

MISRA Compliance on C++: Essentials

Essential MISRA Reading in a C++ context:

Chapter 2: "Disadvantages of C++" - discusses why standards are needed

Chapter 4: Type conversions and casts

Chapter 6: Basic concepts (particularly control flow)

Chapter 8: Declarations and definitions

Chapter 15: Resource management (RAII)

MISRA Compliance on C++: Essentials

Chapter 2: "Disadvantages of C++" - discusses why standards are needed

Undefined Behavior: C++ has many cases where behavior is not specified by the standard (buffer overflows, dereferencing null pointers, signed integer overflow)

Implementation-Defined Behavior: Compiler-specific behaviors that can vary

Unspecified Behavior: Standard allows multiple valid outcomes

Complexity: C++ is a large, complex language with many features that can interact in unexpected ways

Backward Compatibility: Legacy features retained for compatibility can be dangerous in safety-critical contexts

Lack of Built-in Safety: No automatic bounds checking, memory management, or initialization

Standards like MISRA restrict the language to a safer, more predictable subset by eliminating undefined behaviors and dangerous constructs.

MISRA Compliance on C++: Essentials

Chapter 4: Type conversions and casts

Implicit Conversions Are Dangerous: Can lose data silently (int to unsigned, float to int, pointer conversions)

Signed/Unsigned Mixing: Major source of bugs in embedded systems

Narrowing Conversions: Losing precision without warning

Pointer Casts: Violate type safety and can cause alignment issues

MISRA Approach:

- Prefer explicit casts over implicit conversions
- Use `static_cast` for safe conversions
- Avoid `reinterpret_cast` and C-style casts
- Never cast away `const`

MISRA Compliance on C++: Essentials

Chapter 6: Basic concepts (particularly control flow)

Goto Considered Harmful: Makes code difficult to analyze and verify

Single Entry/Exit: Functions should have clear, predictable flow

Loop Bounds: All loops should have deterministic, analyzable bounds

Switch Statements: Must have default cases, no fall-through

Cyclomatic Complexity: Keep functions simple and testable

MISRA Compliance on C++: Essentials

Chapter 8: Declarations and definitions

Initialization Is Mandatory: All variables must be initialized before use

Scope Minimization: Declare variables in smallest possible scope

One Declaration Per Line: Improves readability and reduces errors

Consistent Naming: Clear, meaningful names

Avoid Shadowing: Don't reuse names in nested scopes

MISRA Compliance on C++: Essentials

Chapter 15: Resource management (RAII)

RAII is Fundamental: Resources tied to object lifetime

No Naked new/delete: Use smart pointers

Deterministic Cleanup: Destructors must not fail

Exception Safety: Resources must be released even when exceptions occur

Manager Classes: Scoped, unique, and general managers for resource handling

MISRA Compliance on C++: Violation Categories

Mandatory

Must be followed - No exceptions allowed

Cannot be deviated from under any circumstances

Typically covers critical safety issues or undefined behavior

Violation = Immediate failure in certification

MISRA Compliance on C++: Violation Categories

Required (Most Common)

Should be followed - Deviations require formal justification

Can be deviated from, but you must:

1. Document why you're deviating
2. Provide technical justification
3. Get approval from project safety authority
4. Prove the deviation is safe

Covers important safety/quality issues

MISRA Compliance on C++: Violation Categories

Advisory

Recommended - Should follow when practical

Deviations don't require formal documentation

Best practices that improve code quality

May have performance and/or practical trade-offs

MISRA Compliance on C++: Examples

Disclaimer: the code examples show what pitfalls to avoid but sometimes also introduce other code smells (that is often done for simplicity reasons).

We will check our final example using the MISRA C++ 2023 rules and cppcheck!

Example MISRA C++ Rule 8.4.1: A variable shall be initialized before it is read

```
1 // VIOLATION - Uninitialized variable
2 void badFunction() {
3     int temperature;
4
5     if (sensorReady()) {
6         temperature = readSensor();
7     }
8
9     // BUG: temperature might be uninitialized here!
10    processTemperature(temperature);
11 }
12
13 // COMPLIANT - Always initialized
14 void goodFunction() {
15     int temperature = 0; // Initialize with sensible default
16
17     if (sensorReady()) {
18         temperature = readSensor();
19     }
20
21    processTemperature(temperature);
22 }
```

Example MISRA C++ Rule 8.4.1: A variable shall be initialized before it is read

```
13 // COMPLIANT - Always initialized
14 void goodFunction() {
15     int temperature = 0; // Initialize with sensible default
16
17     if (sensorReady()) {
18         temperature = readSensor();
19     }
20
21     processTemperature(temperature);
22 }
23
24 // BETTER - Initialize at declaration
25 void betterFunction() {
26     if (!sensorReady()) {
27         return; // Early exit if sensor not ready
28     }
29
30     int temperature = readSensor(); // Initialized immediately
31     processTemperature(temperature);
32 }
```

Example MISRA C++ Rule 8.4.1: A variable shall be initialized before it is read

```
1 // Therac-25 had issues like this:
2 uint8_t treatmentMode; // Uninitialized!
3
4 if (operatorChangedMode) {
5     treatmentMode = newMode;
6 }
7
8 // If operator didn't change mode, treatmentMode contains garbage
9 applyRadiation(treatmentMode); // DANGEROUS!
```

Example MISRA C++ Rule 7.1.1:

A variable which is not modified shall be const qualified

```
1 // VIOLATION - Not using const
2 class TemperatureSensor {
3     float threshold;
4
5 public:
6     TemperatureSensor(float t) : threshold(t) {}
7
8     // This function doesn't modify the object but isn't marked const!
9     bool isOverheating(float temp) {
10         return temp > threshold;
11     }
12 };
13
14 void checkSensors(TemperatureSensor* sensor) { // Non-const pointer
15     float reading = 75.5f;
16
17     // Can accidentally modify sensor here!
18     // sensor->threshold = 100.0f; // Oops!
19
20     if (sensor->isOverheating(reading)) {
21         alarm();
22     }
23 }
```

Example MISRA C++ Rule 7.1.1:

A variable which is not modified shall be const qualified

```
25 // COMPLIANT – Proper const usage
26 class TemperatureSensor {
27     const float threshold; // const member – can't be changed after construction
28
29 public:
30     TemperatureSensor(float t) : threshold(t) {}
31
32     // const member function – documents that object isn't modified
33     bool isOverheating(const float temp) const {
34         return temp > threshold;
35     }
36
37     float getThreshold() const { // Getter is const
38         return threshold;
39     }
40 };
41
42 void checkSensors(const TemperatureSensor* sensor) { // const pointer
43     const float reading = 75.5f; // const variable
44
45     // Compiler prevents accidental modification:
46     // sensor->threshold = 100.0f; // ERROR: const object!
47
48     if (sensor->isOverheating(reading)) {
49         alarm();
50     }
51 }
```

Example MISRA C++ Rule 7.1.1:

A variable which is not modified shall be const qualified

```
1 // PRACTICAL EXAMPLE
2 class PIDController {
3     const float kP, kI, kD; // Constants don't change
4     float integral, prevError; // State variables that do change
5
6 public:
7     PIDController(const float p, const float i, const float d)
8         : kP(p), kI(i), kD(d), integral(0.0f), prevError(0.0f) {}
9
10    float compute(const float setpoint, const float measured) {
11        const float error = setpoint - measured; // Local const
12        integral += error;
13        const float derivative = error - prevError;
14
15        prevError = error; // Modify state
16
17        return kP * error + kI * integral + kD * derivative;
18    }
19
20    // const function - just reads state
21    float getIntegral() const {
22        return integral;
23    }
24 };
```

Example MISRA C++ Rule 15.0.1:

A class that appears to manage resources should use RAI

```
1 // VIOLATION - Manual resource management
2 class DataLogger {
3     FILE* logFile;
4
5 public:
6     DataLogger(const char* filename) {
7         logFile = fopen(filename, "w");
8     }
9
10    void log(const char* message) {
11        if (logFile) {
12            fprintf(logFile, "%s\n", message);
13        }
14    }
15
16    // PROBLEM: User must remember to call close()!
17    void close() {
18        if (logFile) {
19            fclose(logFile);
20            logFile = nullptr;
21        }
22    }
23 };
24
25 void buggyCode() {
26     DataLogger logger("system.log");
27     logger.log("Starting...");
28
29     if (errorCondition()) {
30         return; // BUG: File never closed! Memory leak!
31     }
32
33     logger.log("Done");
34     logger.close(); // Only closed if we get here
35 }
```

Example MISRA C++ Rule 15.0.1:

A class that appears to manage resources should use RAI

```
37 // COMPLIANT - RAI pattern
38 class DataLogger {
39     FILE* logFile;
40
41 public:
42     // Constructor acquires resource
43     DataLogger(const char* filename) : logFile(nullptr) {
44         logFile = fopen(filename, "w");
45         if (!logFile) {
46             throw std::runtime_error("Failed to open log file");
47         }
48     }
49
50     // Destructor releases resource automatically
51     ~DataLogger() {
52         if (logFile) {
53             fclose(logFile);
54             logFile = nullptr;
55         }
56     }
57
58     // Prevent copying (would cause double-free)
59     DataLogger(const DataLogger&) = delete;
60     DataLogger& operator=(const DataLogger&) = delete;
61
62     void log(const char* message) {
63         if (logFile) {
64             fprintf(logFile, "%s\n", message);
65         }
66     }
67 };
68
69 void correctCode() {
70     DataLogger logger("system.log");
71     logger.log("Starting...");
72
73     if (errorCondition()) {
74         return; // File automatically closed by destructor!
75     }
76
77     logger.log("Done");
78     // File automatically closed here too
79 }
80
81 // EVEN BETTER - Use standard library RAI
82 void bestCode() {
83     std::ofstream logger("system.log");
84     logger << "Starting...\n";
85
86     if (errorCondition()) {
87         return; // Automatically closed!
88     }
89
90     logger << "Done\n";
91     // Automatically closed
92 }
```

Example MISRA C++ Rule Rule

18.0.2: Do not use new or delete; use smart pointers instead

```
1 // VIOLATION - Raw new/delete
2 void processData() {
3     int* buffer = new int[1000]; // Manual allocation
4
5     readData(buffer, 1000);
6
7     if (errorCondition()) {
8         return; // BUG: Memory leak!
9     }
10
11     processBuffer(buffer, 1000);
12
13     delete[] buffer; // Only freed if we get here
14 }
15
16 class Sensor {
17     int* readings;
18     size_t count;
19
20 public:
21     Sensor(size_t n) : count(n) {
22         readings = new int[n]; // Manual allocation
23     }
24
25     // PROBLEM: If we forget destructor, memory leaks!
26     // PROBLEM: If we copy this object, double-free!
27     ~Sensor() {
28         delete[] readings;
29     }
30 };
```

Example MISRA C++ Rule

18.0.2: Do not use new or delete; use smart pointers instead

```
1 // COMPLIANT - Use std::unique_ptr for single ownership
2 void processData() {
3     std::unique_ptr<int[]> buffer(new int[1000]);
4     // Or better: auto buffer = std::make_unique<int[]>(1000);
5
6     readData(buffer.get(), 1000);
7
8     if (errorCondition()) {
9         return; // Memory automatically freed!
10    }
11
12    processBuffer(buffer.get(), 1000);
13
14    // Memory automatically freed here too
15 }
16
17 // BETTER - Use std::vector (preferred for arrays)
18 void processDataBetter() {
19     std::vector<int> buffer(1000);
20
21     readData(buffer.data(), buffer.size());
22
23     if (errorCondition()) {
24         return; // Automatically cleaned up!
25     }
26
27     processBuffer(buffer.data(), buffer.size());
28 }
```

Example MISRA C++ Rule Rule

18.0.2: Do not use new or delete; use smart pointers instead

```
30 // COMPLIANT - Smart pointer in class
31 class Sensor {
32     std::unique_ptr<int[]> readings;
33     size_t count;
34
35 public:
36     Sensor(size_t n) : readings(std::make_unique<int[]>(n)), count(n) {}
37
38     // No need for explicit destructor - unique_ptr handles it!
39     // Copying is automatically prevented by unique_ptr
40
41     int getReading(size_t index) const {
42         if (index < count) {
43             return readings[index];
44         }
45         return 0;
46     }
47 };
```

Example MISRA C++ Rule Rule

18.0.2: Do not use new or delete; use smart pointers instead

```
49 // BEST – Use std::vector in class
50 class SensorBetter {
51     std::vector<int> readings;
52
53 public:
54     explicit SensorBetter(size_t n) : readings(n, 0) {}
55
56     // No destructor needed!
57     // Copyable and moveable automatically if needed
58
59     int getReading(size_t index) const {
60         if (index < readings.size()) {
61             return readings[index];
62         }
63         return 0;
64     }
65
66     size_t size() const { return readings.size(); }
67 };
68
```

Example MISRA C++ Rule Rule

18.0.2: Do not use new or delete; use smart pointers instead

```
69 // ALTERNATIVE std::shared_ptr for shared ownership
70 class DataCache {
71     std::shared_ptr<std::vector<int>> data;
72
73 public:
74     DataCache() : data(std::make_shared<std::vector<int>>(1000)) {}
75
76     // Multiple objects can share this data safely
77     std::shared_ptr<std::vector<int>> getData() const {
78         return data; // Reference count increases
79     }
80 }; // Reference count decreases, memory freed when last owner destroyed
```

MISRA Compliance on C++:

Key Rules:

- Rule 8.4.1 (uninitialized variables)
- Rule 8.4.2 (no goto)
- Rule 7.1.1 (const correctness)
- Rule 15.0.1 (RAII)
- Rule 18.0.2 (no raw new/delete)

MISRA Violations Hunt

```
1 // How many MISRA violations?
2 #define MAX_SENSORS 10
3
4 class SensorArray {
5     int* data;
6     int size;
7
8 public:
9     SensorArray(int s) {
10         size = s;
11         data = new int[size];
12     }
13
14     int getValue(int index) {
15         return data[index];
16     }
17
18     void process() {
19         int result;
20         for (int i = 0; i <= size; i++) {
21             if (data[i] < 0)
22                 goto error;
23             result = data[i] * 2;
24         }
25         return;
26
27     error:
28         result = -1;
29     }
30 };
```

```
32 void main() {
33     SensorArray sensors(MAX_SENSORS);
34     sensors.process();
35 }
```



<https://gist.github.com/jakorten/e9ffce1eb57fda54c74704e4bac91cd6>

MISRA Compliance on C++:

```
Scanning directory: Example
✓ Found 1 C++ files

✓ Loaded 146 MISRA C++ rules
[1/1] Checking our_wonderful_code.cpp... 13 violations

Scan Complete!
  Total violations: 13
  Files with issues: 1/1

✓ Report saved to report_example.md

Summary by Priority:
  ● Required: 8
  ● Advisory: 5
```

Tip: in the Repo is also “MoreCode_Anonymized” you could also try to check for violations.

Note: it is very, very likely to find violations in ALL our (more serious) code, that is not the point!!!

More on (static) code checking:

MISRA Checking Quick Reference

Free Tools (Use in practice)

- cppcheck --enable=all mycode.cpp
- clang-tidy mycode.cpp -- -std=c++17
- gcc/clang with: -Wall -Wextra -Wpedantic -Werror

What They Check (MISRA-like)

- ✓ Uninitialized variables
- ✓ Memory leaks
- ✓ Null pointer dereferences
- ✓ Buffer overflows
- ✓ Dead code
- ✓ Complexity metrics

What They Miss (Need commercial tools)

- x Full MISRA rule compliance
- x Certification-grade reports
- x Deep flow analysis

How do we check our code...

CPPCheck

[cpplint](#) to check [Google C++ Style Guide](#) compliance

and other static code analyzers

Remember: they are just one of many in our bag of tools.

More on this next week!

Clang-tidy

clang-tidy Example/our_wonderful_code.cpp

```
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:21
:25: warning: The left operand of '<' is a garbage value [clang-analyzer-core.UndefinedBinaryOperatorResult]
21 |         if (data[i] < 0)
    |             ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:33
:17: note: Calling constructor for 'SensorArray'
33 |     SensorArray sensors(MAX_SENSORS);
    |     ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:11
:16: note: Storing uninitialized value
11 |     data = new int[size];
    |     ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:33
:17: note: Returning from constructor for 'SensorArray'
33 |     SensorArray sensors(MAX_SENSORS);
    |     ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:34
:5: note: Calling 'SensorArray::process'
34 |     sensors.process();
    |     ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:28
:14: note: 'i' initialized to 0
20 |     for (int i = 0; i <= size; i++) {
    |             ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:28
:9: note: Loop condition is true. Entering loop body
20 |     for (int i = 0; i <= size; i++) {
    |             ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:21
:25: note: The left operand of '<' is a garbage value
21 |         if (data[i] < 0)
    |             ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:23
:13: warning: Value stored to 'result' is never read [clang-analyzer-deadcode.DeadStores]
23 |         result = data[i] * 2;
    |         ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:23
:13: note: Value stored to 'result' is never read
23 |         result = data[i] * 2;
    |         ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:28
:9: warning: Value stored to 'result' is never read [clang-analyzer-deadcode.DeadStores]
28 |         result = -1;
    |         ^
/Users/jakorten/Library/Mobile Documents/com~apple~CloudDocs/Desktop/MISRA/Example/our_wonderful_code.cpp:28
:9: note: Value stored to 'result' is never read
28 |         result = -1;
    |         ^
jakorten@MacBook-Pro-van-JA-3 MISRA % clang-tidy Example/our_wonderful_code.cpp
```

Readability

Clean code is simple and direct.

Clean code reads like well-written prose.

Clean code never obscures the designer's intent but rather is full of crisp abstractions and straightforward lines of control.”

- Grady Booch author of Object Oriented Analysis and Design with Applications (Booch, Jacobson and Rumbaugh were behind UML)

Kent Beck: Order matters!

Code is written once but read hundreds of times. Write for the hundreds of times.” - Kent Beck

- First: Make it WORK (on target hardware; correctness)
- Second: Make it RIGHT (safe, maintainable)
- Make it FIT (within memory constraints)
- Make it FAST (meet real-time deadlines)

Kent Beck: Habits

"I'm not a great programmer; I'm just a good programmer with great habits."

The habits:

- ✓ Write tests
- ✓ Refactor continuously
- ✓ Name things carefully
- ✓ Keep it simple
- ✓ Make it readable

Trade-offs: why *software engineering* vs programming or computer science

Exercise 1: "Spot the Issues"

- Give students the bad code, ask them to find:
- 5 MISRA violations
- 5 code smells
- Which ones overlap?

Exercise 2: "Prioritize the Fixes"

- Which would you fix first? Why?
- Safety vs maintainability trade-offs

Exercise 3: "Refactor Step-by-Step"

- Start with bad code
- Fix MISRA violations
- Remove code smells
- Compare: did fixing one help the other?

Trade-offs: why *software engineering* vs programming or computer science

Combat "vibe coding": Emphasize that these aren't arbitrary rules - each has a failure case. Show real bugs that resulted from violations.

Make it embedded-relevant: Use examples from Arduino/microcontroller context they know - ISRs, hardware registers, timing-critical code.

Code style: Naming Conventions

C++ Code Style Best Practices

Classes: PascalCase (e.g., SensorController, MotorDriver)

Functions/methods: camelCase (e.g., readSensor(), calculateSpeed())

Variables: camelCase (e.g., sensorValue, targetPosition)

Constants: UPPER_SNAKE_CASE (e.g., MAX_SPEED, DEFAULT_TIMEOUT)

Member variables: prefix with m_ (e.g., m_currentState, m_sensorData)

Code style: Code Quality

Always use const for values that don't change

Prefer references (&) over pointers when ownership isn't transferred

Initialize variables at declaration

Avoid magic numbers - use named constants

Write self-documenting code with clear names

(And of course the rest that we already discussed)

Code style: Comments (later workshop)

Explain why, not what (code shows what)

Document complex algorithms and hardware interactions

Keep comments up-to-date with code changes

Code smells

```
python3 code_smell_scanner.py MoreCode_Anonymized
```

```
Scanning 6 C++ files...
```

CODE SMELL ANALYSIS REPORT

Summary:

Total code smells: 99

Files with issues: 6

By Severity:

● Critical : 1 (1.0%)

● Major : 9 (9.1%)

● Minor : 89 (89.9%)

By Type:

- Magic Number : 89 (89.9%)

- Dead Code : 5 (5.1%)

- Duplicate Code : 4 (4.0%)

```

• God Class      : 1 ( 1.0%)

```

Top Recommendations:

● **CRITICAL:** Address Large/God classes immediately

- MAJOR: Eliminate duplicate code through abstraction

- MINOR: Replace magic numbers with named constants

Code style

Of course companies might have their own style guidelines with do's and don'ts!

Code style at file level

Kolpackov's "Canonical Project Structure" (ISO C++ Paper P1204R0, 2018) argues that split layouts make "navigating between corresponding headers and sources cumbersome" and that "in practice, private headers are placed into include/, defeating the purpose"

Code style: mixing (natural) languages

Sutter & Alexandrescu (2005): Coding standards promote consistency and international collaboration
Micro-os-plus

Google C++ Style Guide: Emphasizes clear, unambiguous naming for global readability Google
JSF AV C++ (Safety-critical standard): Consistent, descriptive English identifiers required Bjarne
Stroustrup

Bottom Line: For embedded systems that may be maintained internationally or in professional contexts, English identifiers are mandatory. Dutch variable names create barriers to code review, collaboration, and professional development.

Recommended Refactoring:

cpp	
<i>// Current (Dutch)</i>	<i>→ Recommended (English)</i>
algemeen/	→ common/ or general/
dataPakket.h	→ DataPacket.h
algdef.h	→ CommonDef.h or GeneralDef.h

Project Structure Recommendation:

```
rgbData/  
  RGBData.h  
  RGBData.cpp  
common/                                // English equivalent  
  DataPacket.h  
  DataPacket.cpp  
  CommonDef.h  
  CommonDef.cpp
```

Take away message

Starting today, write code that is:

- ✓ Safe (MISRA)
- ✓ Maintainable (No smells)
- ✓ Readable (Clear intent)

Don't just make it work. Make it RIGHT.

[https://github.com/AEAAEmbedded/
ESE_PROG/tree/main/Workshops](https://github.com/AEAAEmbedded/ESE_PROG/tree/main/Workshops)

Great Resources

C++ Core Guidelines (Stroustrup et al., ISO C++ Standards Committee)

Sutter & Alexandrescu (2005): C++ Coding Standards: 101 Rules, Guidelines, and Best Practices

JSF AV C++ (Lockheed Martin, 2005): Safety-critical embedded systems standard (221 rules)

MISRA C++:2023: Automotive/medical device coding guidelines for critical systems

Google C++ Style Guide: Industry best practices for large-scale development

Kolpackov's "Canonical Project Structure" (ISO C++ Paper P1204R0, 2018)

Sutter & Alexandrescu (2005): Coding standards promote consistency and international collaboration

“

That's all...

Any questions?